

附件 3

西安电子科技大学

智能系统专业实验 课程实验报告

实验名称 实验 1-数据回归

XXXXXXXX 学院 2320xxx 班

姓名 XXXXXX 学号 XXXXXXXXXXXX

同作者 XXXXXXXXXXX

实验日期 202x 年 x 月 x 日

成 绩

实验报告内容基本要求及参考格式

- 一、实验目的
- 二、实验环境
- 三、实验基本原理及步骤（模型描述、理论描述或方案描述）
- 四、实验自主创新（或改进）部分的描述
- 五、实验结果与分析（需给出结果图或表，并进行分析）
- 六、实验的收获及心得体会（或遇到的问题，如何解决）
- 七、实验代码

一、实验目的

1. 理解并掌握利用神经网络解决数据回归问题的基本流程。
2. 学会使用 PyTorch 框架构建全连接神经网络，完成对带噪声数据的拟合。
3. 掌握训练过程中的动态可视化方法，能够观察拟合曲线随迭代次数的变化。

二、实验环境

1. **硬件**: SUB KIT NX 嵌入式开发板
2. **操作系统**: Linux
3. **软件及库**:
 - 1) Python 3.9
 - 2) NumPy 1.24
 - 3) PyTorch 2.0
 - 4) Matplotlib 3.7

三、实验基本原理及步骤（模型描述、理论描述或方案描述）

1. 基本原理

数据回归是指给定输入 x 带噪声的目标输出 y ，寻找一个函数 $f(x)$ 使得 $f(x)$ 尽可能逼近真实的 y 。本实验采用三层全连接神经网络（输入层→隐藏层→输出层）来拟合非线性函数。

- 1) 激活函数: ReLU，引入非线性能力。
- 2) 损失函数: 均方误差 (MSE)，衡量预测值与真实值的差距。
- 3) 优化算法: 随机梯度下降 (SGD)，通过反向传播更新网络权重。

2. 实验步骤

1) 生成样本数据

在区间 $[-\pi, \pi]$ 内均匀生成 100 个点作为 x ，对应的真值为 $y = \sin(x)$ ，并添加服从 $[0,1]$ 均匀分布的噪声（噪声系数 0.5）。

2) 构建神经网络

- a) 输入维度: 1
- b) 隐藏层: 10 个神经元, ReLU 激活
- c) 输出维度: 1
- d) 使用 `nn.Sequential` 封装。

3) 训练网络

- a) 优化器: SGD, 学习率 0.1
- b) 损失函数: MSELoss
- c) 迭代次数: 10000 次

4) 动态可视化

- a) 每 1000 个 epoch 刷新一次图像:
- b) 用散点图绘制原始数据
- c) 用蓝色绘制当前拟合曲线
- d) 显示当前 epoch 和损失值

5) 训练结束后

关闭交互模式并保持最终图像显示。

四、实验自主创新（或改进）部分的描述

1. 修正原始数据函数

原实验指导中 $y = \text{torch.cos}(x) + 0.3 * \text{torch.rand}(x.size())$, 本实验改为 $y = \text{torch.sin}(x) + 0.5 * \text{torch.rand}(x.size())$ 。

2. 调整噪声幅度

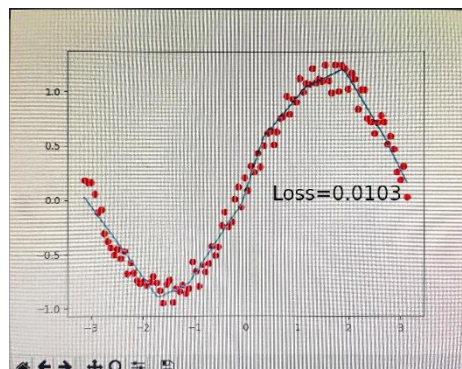
将噪声系数由指导书中的 0.3 增大到 0.5, 使数据更离散, 增加了回归难度, 更考验模型的泛化能力。

3. 动态显示优化

在图像上同时显示 epoch 序号和当前损失值, 并设置适当的暂停时间, 使训练过程更直观。

五、实验结果与分析（需给出结果图或表，并进行分析）

1. 最终拟合效果图



- 1) **散点**: 红色散点分布在正弦曲线周围, 垂直分散程度受噪声影响。
- 2) **拟合曲线**: 蓝色实线呈正弦形, 与真实 $\sin(x)$ 高度一致, 在峰值附近略有平滑差异。
2. **损失变化**: 失函数平稳下降, 未出现震荡或发散, 说明学习率选择合适, 网络结构能够表达目标函数。最终损失稳定在 0.0103。

六、实验的收获及心得体会 (或遇到的问题, 如何解决)

1. **掌握 PyTorch 基本流程**: 数据准备→网络定义→损失与优化器→训练循环→可视化, 这一套流程可复用于许多深度学习任务。
2. **理解噪声对回归的影响**: 过强的噪声会掩盖真实信号, 本实验通过适度噪声让网络学到主要趋势而非过拟合每个点。
3. **动态调试的重要性**: 通过实时观察拟合曲线, 可以快速判断欠拟合 (曲线太平滑) 或过拟合 (曲线剧烈震荡) 现象, 并及时调整网络容量或学习率。

七、实验代码

代码

```
import numpy as np
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

# 生成样本数据: 正弦曲线 + 噪声
x = torch.unsqueeze(torch.linspace(-np.pi, np.pi, 100), dim=1)
y = torch.sin(x) + 0.5 * torch.rand(x.size()) # 噪声系数 0.5

# 定义网络结构 (1-10-1, ReLU)
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.predict = nn.Sequential(
            nn.Linear(1, 10),
            nn.ReLU(),
            nn.Linear(10, 1)
        )
    def forward(self, x):
        return self.predict(x)
```

```

net = Net()
optimizer = torch.optim.SGD(net.parameters(), lr=0.1)
loss_func = nn.MSELoss()

# 动态可视化训练过程
plt.ion()
for epoch in range(10000):
    out = net(x)
    loss = loss_func(out, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 1000 == 0:
        plt.cla()
        plt.scatter(x.data.numpy(), y.data.numpy(), s=10, c='b',
alpha=0.6)
        plt.plot(x.data.numpy(), out.data.numpy(), 'r-', lw=2)
        plt.text(-2.5, 1.2, f'Epoch = {epoch}, Loss =
{loss.item():.4f}',
                fontdict={'size': 10, 'color': 'red'})
        plt.xlabel('x')
        plt.ylabel('y')
        plt.title('Regression with Neural Network')
        plt.pause(0.5)

plt.ioff()
plt.show()

```